Sriram Venkataraman

Lead / Staff Quality Engineer — Test Automation Frameworks & AI Quality Engineering

Email: sriram20mail@gmail.com | Phone: +44 7459 606294 | Location: Edinburgh, UK | Remote (GMT) | LinkedIn: www.linkedin.com/in/SriramVenkataramanlkd | Portfolio: sriramvenkataraman-portfolio.vercel.app

Full right to work in the UK — no sponsorship required

Professional Summary

Lead quality engineer with 16 years in the discipline, the last five building the TypeScript automation frameworks, CI pipelines and quality gates that 21 engineering teams ship against across web, iOS and Android — Cypress on web, WebdriverIO/Appium and Detox on native, Postman and SuperTest on the API layer. I define the test-layer model teams work to, using the testing pyramid as the default shape and the trophy or honeycomb where the architecture warrants it, so coverage lands where it is cheapest and E2E stays on critical journeys. I own the test strategy, quality gates, exit criteria and release-readiness signals for 21 teams across 3 pillars and 3 geographic locations with no direct reports — adoption earned by embedding with squads, running a QA Community of Practice and testing-efficiency audits, and giving go/no-go input at release time. Hands-on with agentic quality workflows rather than AI code generation alone: reusable multi-agent skills with bounded responsibility and human review, plus natural-language test authoring across web and native.

What I Bring

Framework ownership, end to end	Designing, building and evolving TypeScript automation frameworks engineers actually adopt — page/screen-object architecture, conventions, reusable helpers, onboarding docs and the CI that runs them — across web, native and API layers.
AI-augmented and agentic quality workflows	Reusable multi-agent skills that triage regressions and diagnose failing suites, with clear responsibilities, isolated execution and human review — not simply prompting an IDE to generate test code. Natural-language test authoring over a reusable prompt library.
Pragmatic AI adoption in regulated environments	Deterministic LLM caching for cost control and repeatability, false-positive discipline (sub-2% on a CI gate), least-privilege cloud access for model infrastructure, and human sign-off on anything that changes a repository.
Test coverage strategy across test-layer models	Defining which layer a test belongs at — testing pyramid as the default, trophy and honeycomb where component or service architecture warrants — and driving new coverage into the right layer instead of defaulting to E2E; coverage-gap audits and cross-layer coverage mapping.
Unit, component, API and integration testing	Jest and React Testing Library component tests, Postman and SuperTest API suites, contract testing, and service virtualisation against stubbed, containerised environments (Mountebank, WireMock, Docker).
Quality gates that hold	PR-blocking gates on coverage, maintainability, security and performance; exit criteria and release-readiness signals; actionable failure reporting rather than red builds teams ignore.
CI fast enough that engineers keep it in the loop	Test parallelisation and sharding, build sharing and git-aware caching, Terraform-provisioned AWS infrastructure — ~70% faster test runs and 180m→30m test builds.
Quality observability and non-functional breadth	Datadog, Sentry, SLOs and error budgets, statistical change-point detection for performance regressions; accessibility regression automation; security and reliability gates; POC-led tool evaluation before adoption.
Influence without authority	Coaching engineers on test design and the testing pyramid, running a QA Community of Practice, hiring quality engineers, and defending quality in go/no-go discussions across 21 teams.

Key Impact

Rapid feedback enablement	Cut native test runs ~70% and test-build time ~80% (180m→30m) through 4-way Buildkite/Jest sharding, build sharing and a hybrid git-aware cache — keeping CI inside the developer feedback loop rather than a step engineers learn to route around.
Org-wide code quality lift	Unit-test coverage across the codebase improved from 55% to 70% and duplication cut from 45% to 20% by introducing PR-blocking SonarCloud quality gates — coverage earned at unit and component level, not by adding more E2E.
Performance regressions caught before they shipped	A Python E-Divisive + Datadog change-point detector gating CI at under 2% false positives, so the alerts stayed trusted and acted on instead of being muted — the difference between a gate and noise.
Systematic quality improvement at org scale	One Quality Maturity Model adopted across 21 teams, 3 pillars and 3 geographic locations, giving every squad the same quality gates, exit criteria and release-readiness bar instead of 21 local interpretations — a consistent route to shipping faster without trading product quality for speed.
Silent failures turned into automated signal	A QA-environment smoke framework that surfaced 76+ recurring failures across five categories over six months — previously dependent on someone noticing.

New AI capability introduced, not merely adopted	Reusable multi-agent skills for performance triage and E2E failure diagnosis, now used beyond my own team, plus natural-language test authoring across web and native — lowering the cost of writing and maintaining coverage.

Core Skills

Automation Frameworks & Languages: TypeScript, JavaScript, Python, Bash, SQL, Cypress, WebdriverIO, Selenium WebDriver, Protractor, Detox, Appium, Maestro, Jest / React Testing Library, Cucumber / Gherkin (BDD), Page-object & screen-object architecture

Test Strategy & Coverage: Testing pyramid, trophy and honeycomb models, Test-layer rationalisation, Coverage strategy & cross-layer coverage mapping, Risk-based testing, Shift-left / defect prevention, Unit / component / API / integration testing, Contract testing, Test data strategy, Exploratory testing, Flaky-test detection

AI & Agentic Quality Engineering: Claude / Claude Code, Cursor, GitHub Copilot, Multi-agent orchestration, Authoring reusable agent skills, Spec-first plan-and-implement workflows, Human-in-the-loop agent design, Natural-language test authoring (wix-pilot, cy.prompt), LLM prompt libraries, Deterministic caching & cost control, AWS Bedrock

API & Service Testing: Postman, SuperTest, ReadyAPI, REST API automation, Mountebank, WireMock, Swagger-driven mocks, Service virtualisation, Stubbed & containerised pre-integration environments

CI/CD, Cloud & Infrastructure: Buildkite, GitHub Actions, Jenkins, Azure DevOps, Git / GitHub, Test parallelisation & Jest sharding, Build caching, PR quality gates, Docker, Terraform, AWS (S3, IAM, Bedrock, Cognito)

Quality Observability & Non-Functional: Datadog metrics & events, Sentry, SLOs & error budgets, Statistical change-point detection (E-Divisive), Performance-regression detection & baseline validation, k6 load testing, Accessibility testing, SonarCloud security, maintainability & coverage gates, Reliability & smoke monitoring, Visual regression (BrowserStack App Percy), Device-farm management

Leadership & Ways of Working: Quality gates & exit criteria, Release readiness / go-no-go, 3 Amigos & refinement, Coaching & mentoring engineers, POC-led tool evaluation, QA Community of Practice, Distributed & remote teams, Test estimation & planning, Defect management & triage, Regulated & complex domains (financial services, insurance, betting, logistics)

Professional Experience

Lead Quality Engineer — Sportsbook, FanDuel

2022 – Present

Edinburgh, UK

Framework & CI engineering

- Designed and evolved the TypeScript automation frameworks other squads build on — Cypress for Sportsbook web, WebdriverIO/Appium and Detox for iOS and Android (58 specs) — with page and screen-object architecture, reusable helpers, conventions and onboarding docs.
- Extended API-layer coverage with Postman collections and SuperTest suites run in CI, keeping service contracts verified below the UI rather than through E2E journeys.
- Cut test runtime ~70% (4-way Buildkite/Jest sharding) and test-build time ~80% (180m→30m) via build sharing and a hybrid git-aware cache, on Terraform-provisioned AWS S3 with least-privilege IAM.
- Built launch-argument testability into the product (native Swift/Kotlin + React Native module) — runtime environment switching, mock injection, feature-flag overrides and test IDs in production builds — giving deterministic tests with zero rebuilds; reused by another product line.
- Extended that channel to carry feature-flag overrides that survive an Android deeplink relaunch, making deterministic flag-on/flag-off testing the standard across the acceptance and E2E suites.
- Built BrowserStack App Percy visual regression across a 10-device real-device matrix (iOS + Android) for a fast-moving home-page redesign — POC-selected tooling, delivered in ~5 weeks, adopted by 3 squads.
- Own the BrowserStack device-farm estate and the integration patterns other squads plug into, rather than each team standing up its own device-cloud access.
- Built a QA-environment smoke framework automating detection of 76+ recurring failures across five categories.

Test-layer strategy, quality gates & coverage

- Define the test-layer model squads work to — testing pyramid as the default shape, with trophy and honeycomb variants where component and service architecture warrants — and drive new automated coverage into the cheapest layer that can catch the defect rather than defaulting to E2E.
- Introduced SonarCloud PR-blocking quality gates (coverage, maintainability, readability, security) that lifted unit-test coverage 55%→70% and cut duplication 45%→20%, pulling coverage down the pyramid to unit and component level.
- Ran testing-efficiency audits across squads — coverage gaps, flaky-test patterns, tests sitting at the wrong layer, automation opportunities — and turned them into per-team remediation plans.
- Built a Python E-Divisive + Datadog change-point detector that gates CI on performance regressions with under 2% false-positive noise and Slack alerting, on quality observability instrumented in Datadog and Sentry with SLOs and error budgets.

AI-augmented quality

- Built reusable multi-agent skills used beyond my own team: a Performance Impact Review agent that traces a change-point report to the responsible commit range, analyses the diff, adversarially verifies its own findings and produces a remediation plan;

and an Acceptance-Test Debugger that diagnoses failing E2E runs from environment, build config and failure artefacts, self-updating its known-error playbook.
- Designed agent workflows with bounded responsibility and human review — agents assist engineers and never merge their own changes.
- Ran the POC for and integrated natural-language test authoring (wix-pilot with Detox and Appium; Cypress cy.prompt) with a reusable prompt library and a custom handler forcing deterministic LLM cache hits to control cost and non-determinism.

Strategy & influence

- Own the test strategy and shift-left Quality Maturity Model (40+ documents) for 21 teams across 3 pillars and 3 locations with no direct reports — adoption earned by embedding with squads, running a QA Community of Practice and hands-on onboarding.
- Define the quality gates, exit criteria and release-readiness signals the Sportsbook apps ship against, give go/no-go input, and report quality status to senior leadership.
- Lead POC-driven tool evaluation for the org's testing stack, interview and hire quality engineers, and coach engineers on test design and the testing pyramid.

Senior Quality Engineer — Sportsbook, FanDuel

2021 – 2022

Edinburgh, UK

- Authored the Detox acceptance-test framework from scratch — screen-object patterns, conventions and onboarding documentation — and stood up the foundational CI test pipelines and Datadog observability that every later capability was built on.
- Built and maintained Cypress E2E automation for Sportsbook web alongside the native suites, establishing the shared TypeScript conventions used across both.
- Introduced Postman-based API regression coverage to verify service behaviour independently of the UI.

Lead / Senior Test Analyst — Automation, Cognizant Technology Solutions

Jul 2017 – 2021

Edinburgh, UK

- Built UI and API automation frameworks from scratch for Angular web applications in a microservices environment — Protractor–Cucumber in TypeScript, and ReadyAPI/Postman with a custom Groovy assertion library — plus the Azure DevOps pipelines that ran them (Royal London, Resonate Tech).
- Owned a programme-wide UI framework uplift: hardened reliability against flakiness and built failure-analysis reporting into the pipeline.
- Tested microservices against fully stubbed, containerised environments (Mountebank, WireMock, Swagger-driven mocks, Docker) before integration environments existed.
- Delivered a Protractor accessibility-regression framework in Azure CI/CD that cut accessibility regression effort ~40%, plus a UI error-mocking framework that made error paths automatable.
- Shifted testing left into refinement — static-tested requirements with the PO before stories entered the backlog, and wrote the Gherkin standards and CI/CD acceptance scenarios adopted across Test, BA and Dev.
- Trained client teams and peers in test automation and ran automation feasibility analysis and tool selection with business sign-off — widening who could contribute to the suites.
- Fixed minor application bugs with their accompanying unit tests, and diagnosed defects on integrated environments through Splunk log analysis.

Offshore Test Lead / Senior Test Analyst, Cognizant Technology Solutions

Jan 2010 – Jul 2017

Chennai & Coimbatore, India

- Test Analyst through to Offshore Test Lead: functional and regression testing for Travelers and Lincoln Financial from 2010, then led a 7-engineer test team for Voya Financial across five modules from 2014 — capacity planning, work allocation, cross-module knowledge transfer and status reporting.
- Kick-started test automation with a Selenium/TestNG POC and built Excel-macro tooling that automated weekly status reporting and test-data validation.

Personal Projects, Self-Directed Hands-On & Education

- **Self-directed hands-on (personal projects, non-commercial):** Playwright E2E suites, k6 load-test scripts and Jenkins pipelines built on my own projects to stay current with tooling outside my day-to-day commercial stack.
- **Scorebook App:** React Native / Expo + Node.js/Express monorepo built solo with AI assistance — Maestro mobile flows, GitHub Actions CI. A documented experiment to validate AI-first single-person startup claims.
- **Flag Explorer Game:** Vanilla JS multi-level flag quiz with country facts, built for my daughter. Zero dependencies, deployed on Vercel. flag-explorer.vercel.app
- **Birthday RSVP & Memory Share:** Vanilla JS + Vite RSVP site for my daughter's 9th birthday — guest attendance, allergy capture, and host-curated post-party photo sharing. Deployed on Vercel.
- **CricBuds Prediction Game:** Next.js + Neon DB cricket score prediction game for our Edinburgh East of Scotland league group — season-long leaderboard. cricbuds.vercel.app
- **Education:** B.E. Mechanical Engineering, Anna University — 2005 – 2009 · ISTQB Foundation Level Certified